Table of Contents:

1. Overview:

This project presents a real-time air quality monitoring system using the ESP32 microcontroller, integrated with MQ135 and DHT22 sensors. Sensor data is transmitted to Firebase Realtime Database and visualized on a web dashboard deployed via Vercel through a GitHub repository. The system supports Over-the-Air (OTA) updates and sends email alerts when critical environmental thresholds are exceeded.

2. Introduction:

This project aims to design and implement a real-time air quality monitoring system using the **ESP32 microcontroller**. The system continuously monitors the air quality, temperature, and humidity in the environment using the **MQ135** and **DHT22 sensors**, respectively. The system transmits the data to **Firebase Realtime Database** for remote monitoring and alerts the user if any environmental parameter exceeds the predefined thresholds (temperature, or air quality).

Key features include:

- Real-time data collection from the sensors
- Data storage on **Firebase**
- **OTA updates** for firmware maintenance
- **Email alerts** when critical values are reached

3. Project Objectives:

- Monitor temperature, humidity, and air quality
- Store and visualize data in real-time
- Enable remote firmware updates (OTA)
- Send alerts on critical conditions
- Deploy the Website

4. System Architecture:

This section explains the **overall system design** and the communication between different components. The **ESP32** acts as the main controller and coordinates with the **MQ135** and **DHT22** sensors to gather environmental data. The data is then uploaded to **Firebase** and monitored through a web dashboard. The system is also capable of receiving **Over-The-Air (OTA)** updates for firmware updates and sending **email notifications** when critical thresholds are met.

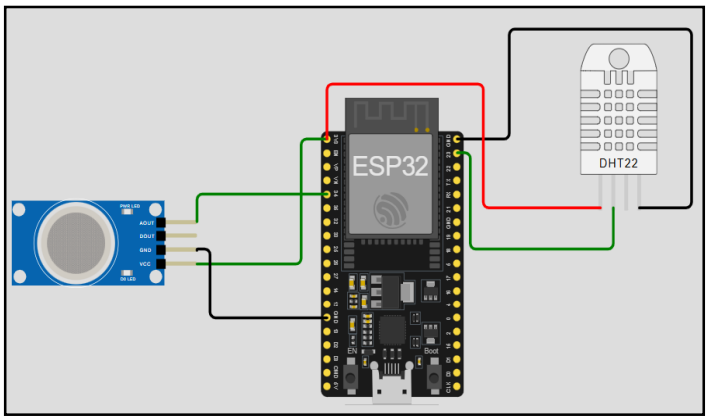
4.1. System Components:

1. **ESP32 Microcontroller:** Handles sensor data collection, Wi-Fi communication, and Firebase interaction.
2. **MQ135 Sensor:** Detects air quality by sensing gases like CO2, NH3, and alcohol vapors.
3. **DHT22 Sensor:** Measures temperature and humidity.
4. **Firebase Realtime Database:** Stores sensor data in real time.

5. Circuit Design and Hardware Setup:

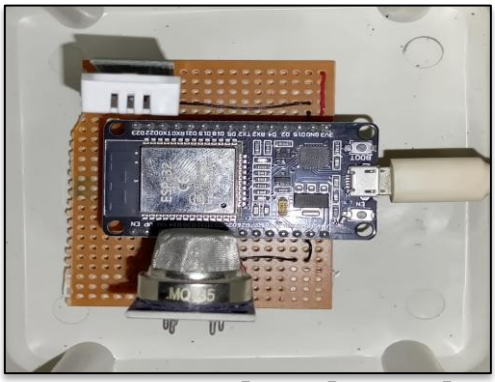
In this project, the **hardware components** are connected to the **ESP32** as follows:

- **DHT22 Sensor:**
  - VCC to 3.3V pin of ESP32
  - GND to ground
  - Data pin to GPIO23 of ESP32
- **MQ135 Sensor:**
  - VCC to 3.3V pin of ESP32
  - GND to ground
  - Analog output pin (A0) connected to GPIO34 of ESP32



5.1. Hardware Setup Explanation:

- The **DHT22** sensor is a **digital sensor** that provides temperature and humidity data.
- The **MQ135** sensor provides an **analog output** that varies based on the air quality, which is then read by the ESP32's **analog-to-digital converter (ADC)**.
- **3D Printed Case:** After testing the circuit on a breadboard, the final setup is transferred to a Vero board with proper soldering. To ensure better rigidity and protection of the components, the circuit is housed in a custom 3D-printed case.



6. Software Design and Code Explanation:

In this section, we break down the software design and explain the functionality of each part of the code.

Software Tools:

- **Arduino IDE**: The development environment used for writing and uploading the firmware to the ESP32.
- **Firebase**: A cloud database to store the sensor data.

- **GitHub**: For managing OTA updates

6.1. Initializing Components and Wi-Fi Setup:

The system first sets up the **Wi-Fi connection** using the provided **SSID** and **password** to allow the ESP32 to communicate with Firebase. Once the Wi-Fi connection is established, the sensors are initialized.

```
void ConnectToWiFi()
{
  Serial.print("Connecting to Wi-Fi...");
  WiFi.begin(SSID, PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConnected to Wi-Fi");
  Serial.println("\n✅ Connected with IP: " + WiFi.localIP().toString());
}
```

6.2. Reading Data from Sensors:

In this section, we explain the process of reading data from the DHT22 and MQ135 sensors, along with the formulas used to process the data and estimate environmental conditions.

6.2.1. Reading Data from the DHT22 Sensor:

The DHT22 sensor provides temperature and humidity data. The ESP32 reads the values using a digital signal.

```
temperature_C = dht.readTemperature();
temperature_F = dht.readTemperature(true);
humidity = dht.readHumidity();
if (isnan(temperature_C) || isnan(humidity) || isnan(temperature_F)) {
  Serial.println("Failed to read from DHT sensor!");
  return;
}
```

These values are then stored for further processing or display.

6.2.2. Reading Data from the MQ135 Sensor:

The MQ135 sensor is used for detecting air quality by measuring the concentration of various gases, particularly CO2. The sensor outputs an analog signal, which the ESP32 reads using its analog-to-digital converter (ADC). The formulae below explain how the raw data is processed:

6.2.2.1. Voltage Calculation from ADC Reading:

The analog output from the MQ135 sensor is read by the ESP32's ADC pin. The ADC value ranges from 0 to 4095 (for a 12-bit ADC). This value corresponds to a voltage between 0 and 3.3V (reference voltage of ESP32).

The voltage is calculated using the formula:

$$\text{voltage} = \text{rawADC} \times \left(\frac{3.3}{4095.0}\right)$$

Where:

- rawADC is the value read by the ADC from the MQ135 sensor (ranging from 0 to 4095).
- 3.3 is the reference voltage provided to the sensor (in volts).
- 4095.0 represents the maximum possible ADC value (12-bit ADC).

This formula converts the raw ADC value into a corresponding voltage.

6.2.2.2. Resistance Calculation (Rs):

The next step is to calculate the resistance of the sensor (Rs). The resistance value is critical for estimating the concentration of gases. The formula to calculate the sensor's resistance is:

$$R_s = R_L \times \left(\frac{V_{cc} - V_{out}}{V_{out}} \right)$$

Where:

- R_S is the sensor resistance (measured in ohms).
- R_L is the load resistance (typically 10kΩ).
- V_{cc} is the supply voltage to the sensor (3.3V in this case).
- V_{out} is the voltage output from the sensor (calculated in the previous step).

6.2.2.3. CO_2 Concentration Estimation (ppm):

Once the sensor resistance (R_S) is calculated, it can be used to estimate the CO_2 concentration in the air. The MQ135 datasheet provides a formula that correlates the sensor resistance with the CO_2 concentration in parts per million (ppm):

$$\text{ppm} = 116.6020682 \times \left(\frac{R_s}{R_0} \right)^{-2.769034857}$$

Where:

- ppm is the estimated concentration of CO2 in parts per million.
- R_S is the sensor resistance calculated in the previous step.
- R_0 is the sensor resistance in clean air (calibration value for the sensor, typically around 76000Ω).

This formula estimates the CO2 concentration based on the ratio of the sensor's resistance (R_s) in polluted air to the reference resistance (R_0) in clean air. The exponent -2.769034857 is derived from the calibration data provided in the MQ135 datasheet.

6.2.3. Periodic Data Collection:

The ESP32 collects sensor data periodically to ensure that the measurements are up-to-date. The data collection occurs in regular intervals, ensuring continuous monitoring of temperature, humidity, and air quality.

6.2.4. Error Handling:

To ensure reliable data collection, error handling is implemented in case the sensors fail to provide valid readings. If an error is detected (e.g., sensor failure), the system logs the error and retries the data collection.

6.3. Uploading Data to Firebase:

Once the data is collected, it is uploaded to **Firebase Realtime Database** for remote monitoring and storage.

```
// Upload DHT22 data to Firebase
FirebaseJson dhtSensorData;
dhtSensorData.set("temperature(°C)", temperature_C);
dhtSensorData.set("temperature(°F)", temperature_F);
dhtSensorData.set("humidity", humidity);
dhtSensorData.set("timestamp", timestamp);
```

```
// Upload MQ-135 data to Firebase
FirebaseJson mq135SensorData;
mq135SensorData.set("MQ135_raw_ADC", analogRead(MQ135_PIN));  // Raw ADC value
mq135SensorData.set("MQ135_voltage", analogRead(MQ135_PIN) * (3.3 / 4095.0));
 mq135SensorData.set("resistance_Rs", rs);
mq135SensorData.set("estimated_CO2_ppm", ppm);
mq135SensorData.set("MQ135_air_quality_status", getAirQualityStatus(ppm));  //
mq135SensorData.set("timestamp", timestamp);
```

6.4. OTA Updates:

The system is designed to check for **firmware updates** periodically. If a new firmware version is available, it will be downloaded and applied using **OTA updates**.

```
void CheckForNewUpdate()
{
  HTTPClient http;
  http.begin(GitHubJsonFile);
  http.addHeader("Authorization", "Bearer " GitHubPAT);
  Serial.println("Downloading JSON file...");
  int httpCode = http.GET();
  if (httpCode == HTTP_CODE_OK) {
    String payload = http.getString();
    Serial.println("JSON payload:");
    Serial.println(payload);
    DynamicJsonDocument doc(512);
    DeserializationError error = deserializeJson(doc, payload);
    if (error) {
      Serial.print("Failed to parse JSON: ");
      Serial.println(error.c_str());
      return;
    }
    firmwareVersion = doc["version"].as<String>();
    updateFileUrl = doc["update_file_url"].as<String>();
    Serial.println("Parsed data:");
    Serial.printf("Firmware Version: %s\n", firmwareVersion.c_str());
    Serial.printf("Update File URL: %s\n", updateFileUrl.c_str());
    double currentVersionNum = CURRENT_FIRMWARE_VERSION.toDouble();
    double newVersionNum = firmwareVersion.toDouble();
    if (newVersionNum > currentVersionNum) {
      ESP32UpdateFirmware(updateFileUrl);
    } else {
      Serial.println("Application is up to date!");
    }
  } else {
    Serial.printf("Failed to download JSON file, HTTP error code: %d\n", httpCode);
    Serial.printf("Error message: %s\n", http.errorToString(httpCode).c_str());
  }
  http.end();
}
```

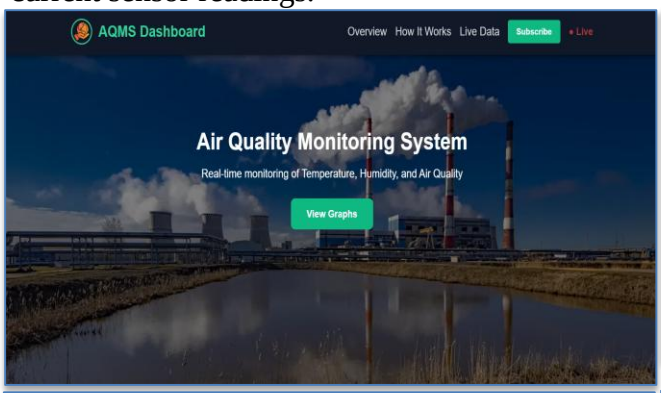
This process ensures that the ESP32 is always running the latest firmware without requiring physical access to the device.

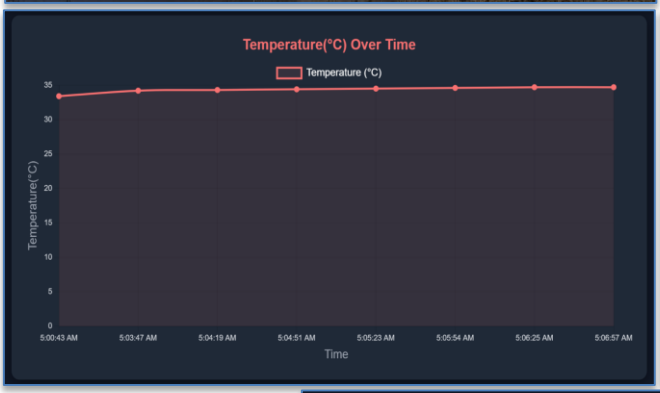
7. Data Visualization, Dashboard, and Website Layout:

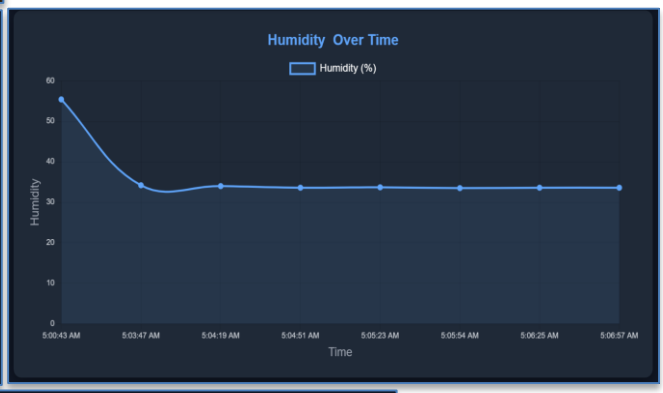
The website layout is designed to be simple and user-friendly, with a focus on real-time data visualization. The homepage displays live sensor data graphs and provides an alert section showing the latest notifications. The system is deployed on **Vercel**, with continuous deployment through a **GitHub repository**, ensuring easy updates.

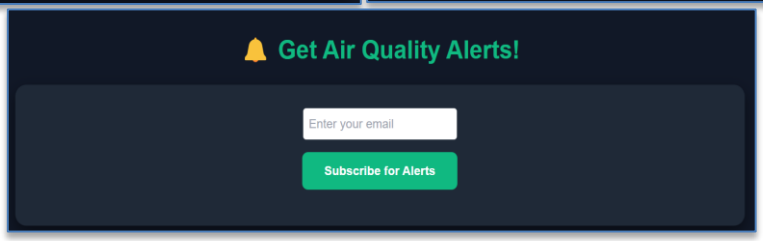
7.1. Dashboard Features:

- **Real-Time Data Visualization**: The dashboard displays live data from the sensors.
- **Graph Display**: The dashboard includes a graph that shows the latest 30 readings from the sensors. This allows users to easily track trends and monitor the environment over time.
- **Alerts**: If the CO2 concentration (PPM) rises above a specified threshold, an alert is triggered to notify users about the potential danger, similar to how gas detectors operate. These alerts are shown on the dashboard in real-time.
- **User Email Notifications:** If the user chooses to subscribe by entering their email during their visit, they receive alerts via email when temperature cross the certain threshold. **Currently, the system does not store the user's email after the session ends**. This means users must re-subscribe each time they visit the site. Once the data is uploaded to **Firebase**, it can be visualized on the web dashboard in real-time. The dashboard continuously updates to reflect the most current sensor readings.









By displaying the live data, users can monitor their environment and receive timely alerts when the air quality deteriorates.

8. Challenges and Solutions:

8.1. Sensor Calibration:

The MQ135 sensor required calibration to accurately detect gases. This was done by adjusting the sensor's output using known air quality values and testing in various environments.

8.2. Wi-Fi Connectivity Issues:

Wi-Fi issues were resolved by repositioning the ESP32 to areas with better signal strength. Ensuring proper placement of the device is crucial for stable operation.

8.3. OTA Update Failures:

Initially, OTA updates failed due to connection issues with the update server. After debugging and ensuring the proper server settings, the OTA functionality worked as expected.

9. Future Enhancements:

The system can be enhanced in several ways:

- **Add More Sensors:** Additional sensors like **CO2**, **PM2.5**, or **VOC sensors** could be integrated to measure more pollutants.
- **Mobile App Integration:** A mobile app could be developed to monitor data on the go.
- **Persistent Email Subscription**: In future versions, the system will store user email addresses in a secure database. This will allow users to subscribe once, and the system will recognize them automatically on future visits (e.g., showing a message like *"You are already subscribed"*).
- **Automatic Control:** The system could be integrated with **smart home devices** to automatically activate air purifiers when air quality drops below a threshold.

10 .Limitations:

Despite its successful deployment and real-time monitoring capabilities, the air quality monitoring system has certain limitations:

- **Wi-Fi Dependency:** The system requires a stable Wi-Fi connection for real-time data upload and dashboard updates. Areas with poor or no Wi-Fi coverage limit the system's effectiveness.
- **Power Supply Stability:** The system depends on continuous power. Power interruptions can cause data loss or disconnection from Firebase until manually restarted.
- **Limited Gas Detection Range:** The MQ135 sensor can detect a variety of gases but does not differentiate between them precisely. It gives a general air quality indication rather than identifying specific pollutants.

- **Latency in OTA Updates:** OTA firmware updates rely on internet speed. In slower networks, firmware updates might experience delays or failures.
- **Data Privacy and Security Risks:** Although Firebase is secure, deploying a web dashboard online can expose vulnerabilities if authentication and database rules are not properly configured.
- **Environmental Factors Affecting Sensors:** Extreme weather conditions, like high humidity or dust accumulation, can affect the accuracy and lifespan of the sensors.

11. Conclusion:

This air quality monitoring system successfully monitors temperature, humidity, and air quality using the **ESP32** and sensors. The system uploads data to **Firebase**, sends email alerts, and can be remotely updated via **OTA**. The system is scalable and can be further improved by adding more sensors or integrating it with a mobile app.

12. References:

- Firebase Realtime Database Documentation - https://firebase.google.com/docs/database
- DHT22 Sensor Datasheet - https://www.microchip.com/wwwproducts/en/DHT22
- MQ135 Sensor Datasheet - https://www.micropik.com/PDF/MQ-135.pdf